Системное программирование

14 января 2026 г. 11:20

BB 11 01 B9 0D 00 B4 0E 8A 07 43 CD 10 E2 F9 CD 20 48
65 6C 6C 6F 2C 20 57 6F 72 6C 64 21

mov
mul
Add

Инструкции

Мнемоники

Записать в регистр EAX число 1 имеет опкод B8
mov eax, 1

Ассемблер - транслятор программы на языке ассемблера в программу на машинном языке

Язык ассемблера - представление команд процессора в виде, доступном для чтения человеком

Компилятор - программа, которая преобразует код на языке программирования в машинный код

Компоновщик - программа, которая производит компоновку или связывание машинного кода

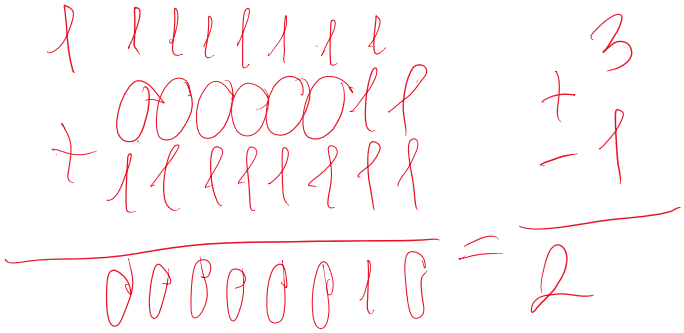
Байт = 256 значений
0b00000000 = 0
0xFF = 255

Для представления отрицательных чисел используется метод дополнения до 2 (two's complement)
0 – 1 = -1
За 0 принимается число 2^n, где n - число битов

0b1<u>00000000</u> = 256
-128~127
256 – 1 = 255 = 0b11111111 = -1
0b00000001
0b11111110
0b11111111

0b00000011 = 3
0b11111100 + 1 =0b11111101 = -3 11111101

0b00000010 = 2
0b11111101 + 1 = 0b11111110 = -2



Основные регистры

14 января 2026 г. 12:15

Регистры - это области высокоскоростной памяти, расположенные внутри процессора в непосредственной близости от его исполнительного ядра

Доступ к ним осуществляется значительно быстрее, чем к оперативной памяти

Регистры общего назначения

AX - Аккумулятор - для арифметических операций

BX – Base - указатель на данные

CX - Счетчик - для хранения счетчиков циклов

DX – Data - для арифметических операций и ввода-вывода

Регистры-указатели

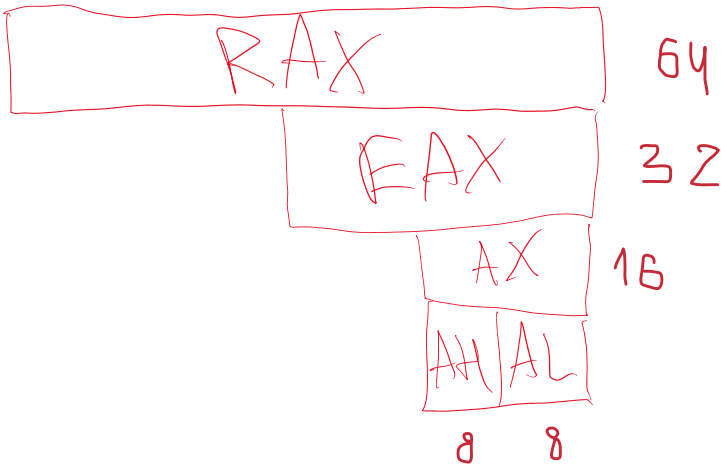
SP - (Stack Pointer) - указатель на стек (вершина стека)

BP - (Base Pointer) - указатель базы стека

SI - (Source Index) - индекс источника

DI - (Destination Index) - индекс назначения

IP - (Instruction Pointer) -указатель инструкции



Дополнительные регистры общего назначения
R8 –R15

Rx – 64
RxD – 32
RxW – 16
RxB – 8

D – dword -двойное слово
W – word - слово
B – byte - байт

Регистр флагов FLAGS (RFLAGS)

0 CF – флаг переноса (Carry flag) - показывает, был ли перенос при сложении или заимствование при вычитании
6 ZF - флаг нуля (Zero flag) - если результат операции равен 0
7 SF - флаг знака (Sign flag) - если результат операции отрицательный
11 OF - флаг переполнения (Overflow flag) - если операция привела к переполнению со знаком

Сегментные регистры
CS - сегмент кода

DS - сегмент данных
SS - сегмент стека
ES, GS, FS - дополнительные сегменты

Инструкции ассемблера NASM

15 января 2026 г. 9:47

mov - копирует данные из одного места в другое

mov приемник, источник

mov eax, 5
;копирует число 5 в регистр eax

mov ebx, eax
;копирует значение из eax в регистр ebx

Значения должны быть одинакового размера

movsx - знаковое расширение
movzx - беззнаковое

mov al, -1
movsx rbi, al

mov al, 5

movzx rbi, al

push - записать значение в стек
push источник

POP - извлечь значение из стека
POP приемник

XCHG - обмен данными

XCHG операнд1, операнд2

Арифметические команды

ADD - сложение
ADD ax, bx ; ax = ax + bx
ADD ax, 5 ; ax = ax + 5

ADC - сложение с переносом (с учетом флага CF)

ADC операнд1, операнд2
операнд1 = операнд1 + операнд2 + CF

SUB - вычитание
SUB ax, bx ; ax = ax - bx
SUB ax, 5 ; ax = ax - 5

SBB - вычитание с заемом (с учетом флага CF)
SBB операнд1, операнд2
операнд1 = операнд1 - операнд2 - CF

Инкремент и декремент
INC операнд ;операнд = операнд + 1
DEC операнд ;операнд = операнд - 1

Умножение
MUL - беззнаковые числа
IMUL - знаковые числа

MUL 5 ; AX = AX*5

MOV EAX, 5

MUL EAX, 3 ; EAX = EAX * 3 = 5*3 = 15

MUL o8	AX = AL*o8
MUL o16	DX:AX = AX*o16
MUL o32	EDX:EAX = EAX*o32
MUL o64	RDX:RAX = RAX*o64

AX - младшая часть
DX - старшая часть

CF и OF - устанавливаются, если результат не поместился AX/EAX/RAX

IMUL операнд
; AX = AX * операнд
IMUL операнд1, операнд2
; операнд1= операнд1 * операнд2
IMUL операнд1, операнд2, операнд3
; операнд1 = операнд2* операнд3

Деление
DIV - беззнаковые числа
IDIV - знаковые числа

DIV o8	AL = AX / o8, ост в AH
DIV o16	AX = DX:AX/o16, ост в DX
DIV o32	EAX = EDX:EAX/o32, ост в EDX
DIV o64	RAX = RDX:RAX/o64, ост в RDX

CBW - преобразует байт в AL в слово в AX (расширение знаком)
CWD - преобразует слово в AX в двойное слово в DX:AX
CDQ - преобразует двойное слово в EAX в четверное слово в EDX:EAX
CQO - преобразует четверное слово в RAX в восьмеричное слово в RDX:RAX

NEG - смена знака числа
NEG операнд

Логические операции

AND операнд1, операнд2

0b00000001 && 0b00000001 = 0b00000001
0b00000011 && 0b00000010 = 0b00000010

OR операнд1, операнд2

XOR операнд1, операнд2

TEST операнд1, операнд2
Выполняет поразрядное И, но меняет флаги ZF, SF, PF

NOT операнд1
Инвертирует биты числа

Команды сдвига

SHL - сдвиг влево

MOV al, 5 ; в al число 00000101
SHL al, 1 ; 00000101 << 1 = 00001010 = 10
Операнд * 2^n

SHR - сдвиг вправо (логический)
MOV al, 5 ; в al число 00000101
SHR al, 1 ; 00000101 >> 1 = 00000010 = 2

Операнд // 2^n

SAR - сдвиг вправо (Арифметический)
MOV al, 5 ; в al число 00000101
SAR al, 1 ; 10000101 >> 1 = 11000010

Дополняет число справа не 0 а значением старшего бита (необходимо для отрицательных)

ROL/ROR - вращение влево/вправо
MOV al, 5 ; в al число 00000101
ROL al, 1 ; 10000101 << 1 = 00001011

RCL/RCR - вращение влево/вправо с переносом, значение записывается в флаг переноса

Команды перехода

Сравнение
CMP операнд1, операнд2
CMP ax, bx

- ax = bx -> ZF
- Если результат отрицательный -> SF
  - ((SF = 0 и OF = 1) или (SF = 1 и OF = 0)) --> AX < BX
  - ((SF = 0 и OF = 0) или (SF = 1 и OF = 1)) -> AX >= BX

JMP - безусловный
JMP метка

label:

JE/JZ - при равенстве/нуле (ZF=1)
JNE/JNZ - при неравенстве/не нуле (ZF=0)
JA/JNBE - первый операнд больше второго (беззнаковые) (CF=0, ZF = 0)
JAE/JNB/JNC - первый операнд больше

или равен второму (беззнаковые) (CF=0)
JB/JNAE/JC - первый операнд меньше второго (беззнаковые) (CF=1)
JBE/JNA - первый операнд меньше или равен второму (беззнаковые) (CF=1 или ZF=1)
JG/JNLE - первый операнд больше второго (SF = OF и ZF = 0)
JGE/JNL - первый операнд больше или равен второму (SF = OF)
JL/JNGE - первый операнд меньше второго (SF != OF)
JLE/JNG - первый операнд меньше или равен второму (SF != OF или ZF = 1)

e	Equal	Равно	Любые
n	Not	Не	Любые
a	Above	Выше	Без знака
b	Below	Ниже	Без знака
g	Greater	Больше	Со знаком
l	Less	Меньше	Со знаком

db – byte - байт
dw – word - слово
dd – dword -двойное слово
dq – qword - четверное слово

Способы адресации

1. Регистровая - значения сохранены в регистр
   mov eax, ecx
2. Непосредственная - операнды непосредственно в коде
   mov eax, 1
3. Косвенная адресация
   [base + (index*scale)+offset]
   Base - базовый регистр
   Index - индексный регистр
   Scale- множитель 1,2,4,8
   Offset - смещение (число) 32-разрядное

[base]	[rdx]
[offset]	[0FFh], [variable]
[base+offset]	[rcx + 0FFh]
[base+index]	[rax+ecx]
[index*scale]	[rcx*4]
[index*scale+offset]	[rcx*4+65]
[base+index*scale]	[rsp+rcx*4]
[base+index*scale+offset]	[rsp+rcx*4+65]

Ветвление с 1 ветвью

```
If(условие){
    действия
}
```

```
CMP опер1, опер2
JCC end_if
    Действия
end_if:
```

Ветвление с 2 ветвями

```
If(условие){
    действия
}
Else{
    действия
}
```

```
CMP опер1, опер2
JCC else_begin
    Действия
JMP end_if
else_begin:
    Действия
end_if:
```

SWITCH-CASE

```
CMP опер, знач1
JE case1
CMP опер, знач2
```

```
JE case2
CMP опер, знач3
JE case3
JMP end_switch
case1:
    Действия
JMP end_switch
case2:
    Действия
JMP end_switch
...
end_switch:


while(условие){
    Действия
}

while_begin:
CMP опер1, опер2
JCC end_while
    Действия
JMP while_begin
end_while:

do{
    Действия
}while(условие)

do_while_begin:
    Действия
```

```
CMP опер1, опер2
JCC end_do_while
JMP do_while_begin
end_do_while:

Int i = 0
While(i<n){
    Действия
    i++
}

LOOP

MOV ecx, 5
MOV eax, 0

mainloop:
    JECXZ exit
    ADD eax, 2
LOOP mainloop
exit:

LOOPE – ZF = 1
LOOPNE – ZF = 0
```

Работа с дробными числами в ассемблере

19 января 2026 г. 12:17

IEEE 754 (float 32b/double 64b)

S (1 бит)	E(8 бит)	M (23 бит)
0-1	00000000-11111111	000...-111...

S – Sign - знак числа 0 - положительное, 1 - отрицательное
E – Exponent - порядок, хранится со смещением (bias = 127 / bias = 1023)
M – Mantissa - мантисса (значащие биты дробной части)

-13,625
S = 1
13 = 1101
*0.625 * 2 = 1.25 (1)*
*0.25 * 2 = 0.5 (0)*
*0.5 * 2 = 1 (1)*
0.625 = 0.101
13,625 = 1101.101
1.101101 * 2^3
M = 10110100000000000000000
E = 3 + bias = 3 + 127 = 130 = 10000010

1 10000010 10110100000000000000000

= $C15A0000_{16} = 3243900928_{10}$

0.2 * 2 = 0.4 (0)
0.4 * 2 = 0.8 (0)
0.8 * 2 = 1.6 (1)
0.6 * 2 = 1.2 (1)
0.2 * 2 = 0.4 (0)

Математический сопроцессор
FPU – floating point unit

Регистры:
Стек процессора - регистры R0-R7 (80 бит)
Служебные (16 бит):
SWR (Status Word Register) - хранит информацию о состоянии сопроцессора
CWR (Control Word Register) - управляет режимами работы сопроцессора
TWR (Tags Word Register) - управляет регистрами R0-R7
Регистры указателей (48 бит):
DPR - указатель данных
IPR - указатель команд

Команды

FINIT - команда инициализации сопроцессора

00000110 00000001 00000110 00000001

	Знач	Вещ	Цел	Десят
Загрузка числа из памяти в стек сопроцессора ST(0)	LoaD	FLD	FILD	FBLD
Сохранение числа из ST(0) в память без извлечения	STore	FST	FIST	FBST
Сохранение числа из ST(0) в память с извлечением	STore + Pop	FSTP	FISTP	FBSTP

FLDZ - загружает 0
FLD1 - загружает 1
FLDPI - загружает число Пи
FLDL2T - загружает двоичный логарифм десяти
FLDL2E - загружает двоичный логарифм экспоненты
FLDLG2 - загружает десятичный логарифм двойки
FLDLN2 - загружает натуральный логарифм двойки

Обмен значения ST(0) с ST(i)
FXCH ST(i) ; i от 1 до 7

Сложение
FADD ; ST(0) =ST(0) + ST(1)
FADD mem ; ST(0) =ST(0) + mem
FADD ST(0), ST(i) ; ST(0) =ST(0) + ST(i)
FADD ST(i), ST(0) ; ST(i) =ST(0) + ST(i)

FADDP - извлекает из стека после операции
FADDP ST(i), ST(0)

Вычитание
FSUB ; ST(0) = ST(1) - ST(0)
FSUB mem ; ST(0) =ST(0) - mem
FSUB ST(0), ST(i) ; ST(0) =ST(0) - ST(i)
FSUB ST(i), ST(0) ; ST(i) =ST(i) - ST(0)

Обратное вычитание
FSUBR - меняет местами при вычитании

Умножение
FMUL

Деление
FDIV

FDIVR - меняет местами при делении

FABS ; ST(0) = |ST(0)|
FCHS ; ST(0) = - ST(0)
FRNDINT ; округление ST(0) до целого, режим в RC
FSQRT ; квадратный корень ST(0)

FSIN - синус ST(0)
FCOS - косинус ST(0)
FPTAN - тангенс ST(0)

FCOM ST(i) ; сравнивает ST(i) с ST(0)

ST(0) > операнд	C0 = 1
ST(0) = операнд	C3 = 1
ST(0) < операнд	C0 = 1 C3 = 1
NaN	C2 = 1

C0-> CF
C2-> PF
C3-> ZF

FSTSW ax - сохранение статусного слова FPU в регистр AX

SAHF - копирует 8 бит AX в регистр флагов

FTST - сравнивает ST(0) с 0

XMM0-XMM15 - регистры для работы с вещественными числами и их хранения

Цепочечные команды

27 января 2026 г. 14:37

Цепочка - непрерывная последовательность байт, слов, двойных слов, четверных слов, обрабатываемых как единое целое

Отличие цепочки от массива: У массива доступ к элементам произвольный, у цепочек - только последовательный

```
section .data
    nums dd 1, 2, 3, 4, 5    ; массив из 5 элементов
    numbers  times 10 dd 2  ;массив из 10 элементов
    2
    buffer resd 5 ; пустой массив из 5 элементов
```

resb, resw, resd, resq

Цепочечные команды - команды для обработки цепочек

Особенность - автоматическое передвижение к следующему элементу цепочки

S – тип данных определяется на основе операнда
B, W, D, Q

RSI/ESI/SI - регистр-источник
RDI/EDI/DI - регистр-приемник

DF (Direction Flag)
DF=0 (CLD) - движение вперед
DF=1 (STD) - движение назад

RCX/ECX/CX (количество повторений)

MOVS адрес_приемника, адрес_источника - пересылка (копирование) данных из одной цепочки в другую

LEA приемник, источник - Load Effective Address - загрузка адреса

/REPE/REPNE

CMPS адрес_приемника, адрес_источника - сравнение данных в цепочках

REP – повтор пока регистр-счетчик не равен 0, автоматически его уменьшает (Команды пересылки и сохранения)

Если вы работаете со строками необходимо в регистр-счетчик записывать значение на 1 больше, чем длина строки

REPE/REPZ - повтор пока регистр-счетчик не равен 0, и ZF = 1
Используется для поиска различающихся элементов

REPNE/REPNZ - повтор пока регистр-счетчик не равен 0, и ZF = 0
Используется для поиска одинаковых элементов

LODS источник - загрузка значение из ESI в EAX

STOS приемник - сохранение значение из EAX в EDI

SCAS приемник - поиск элемента из EAX в EDI

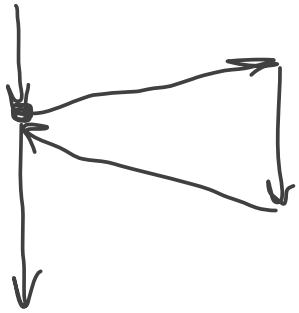
Если необходимо выполнить чтение/запись не с начала цепочки, то требуется увеличить значение в требуемом регистре

ADD ESI, индекс
ADD EDI, индекс

Системные вызовы и прерывания

31 января 2026 г. 12:42

Прерывание (interrupt) - сигнал процессору от ПО или аппаратного обеспечения, что надо прервать выполнение текущего кода, и выполнить то, что указано в обработчике прерывания



1 Асинхронные (внешние, аппаратные) - исходят от внешних устройств и происходят в любой момент времени

2 Синхронные (внутренние) - события в самом процессоре, возникают при нарушении каких-либо условий (деление на 0, переполнение стека)

3 Программные - инициируются в ходе исполнения специальных инструкций в коде программ

int (Interrupt) - инструкция генерирующая программное прерывание

int n ; n - номер прерывания от 0 – 255

int 0x80 - вызов системной функции linux

Системные вызовы - системные функции, приостанавливающие выполнение программы и передающие управление ядру операционной системы, которое выполняет соответствующую функцию и возвращает управление

syscall – выполнение системного вызова

номер вызова помещается в RAX
RDI, RSI, RDX, R10, R8, R9

[Linux syscall table: latest, x86 64-bit, x86-64 ABI (64-bit)]

```
section .data
  msg db "Hello World", 0
  len equ $-msg

section .text
global main

main:
  ;вывод сообщения на экран

  mov eax, 4 ; sys_write
  mov ebx, 1 ; дескриптор стандартного вывода 1 - stdout
  mov ecx, msg; сообщение
  mov edx, len; длина
  int 0x80 ;прерывание на вызов системной функции

  ;завершение программы
  mov eax, 1 ;sys_exit
  xor ebx, ebx ; код возврата - 0
  int 0x80 ;прерывание на вызов системной функции
```

Функции

31 января 2026 г. 13:41

Функция - набор инструкций, которые проектируются на некоторую метку, которая и выступает именем функции. Набор инструкций функции должен завершаться инструкцией ***ret***, которая передает управление вызывающему коду

```
метка:      ;определение функции
    ;инструкции
    ret

call метка ;вызов функции
```

call помещает в стек адрес возврата функции (RSP)
ret извлекает адрес из стека

RAX - регистр для сохранения результата функции

C/C++
Linux – RDI, RSI, RDX, RCX, R8, R9
Windows – RCX, RDX, R8, R9

Взаимодействие кода ассемблера с C/C++

Windows –	RCX, RDX, R8, R9
	XMM0, XMM1, XMM2, XMM3

5 – [RSP+32]
6 – [RSP+40]
И т.д.

void someFunc(int a, double b, char *c, double d)

a	RCX
b	XMM1 и RDX
c	R8
d	XMM3 и R9

Параметры всегда представляют 8-байтовые значения

Необходимо резервировать 32 байта - shadow storage

Выравнивание стека по 16 байтам

Если функция возвращает целое число, оно помещается в RAX, если вещественное - в XMM0

Дизассемблирование

10 февраля 2026 г. 8:38

VisualStudio - Отладка - Дизассемблированный код

HexEdit - редактор файлов в 16-ричном формате

dumpbin /disasm имя_файла.exe > дамп.txt

dumpbin /rawdata имя_файла.exe > данные.txt

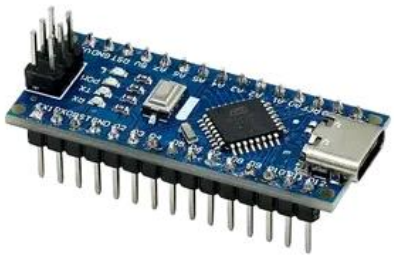
Arduino

12 февраля 2026 г. 12:49

Популярная платформа для разработки встраиваемых систем, включающая микроконтроллеры и программное обеспечение.

IoT - интернет вещей

Arduino Nano



ESP-32

Arduino IDE
Tinkercad
Wokwi

Выполняется при включении устройства

```
void setup() {
  // put your setup code here, to run once:

}
```

Выполняется постоянно пока устройство работает

```
void loop() {
  // put your main code here, to run repeatedly:


}
```

Пин - выход/вход устройства

pinMode(номер_пина, режим);
Режимы: OUTPUT/INPUT/INPUT_PULLUP

digitalWrite(номер_пина, значение);
Значения: HIGH/LOW

digitalRead(номер_пина);

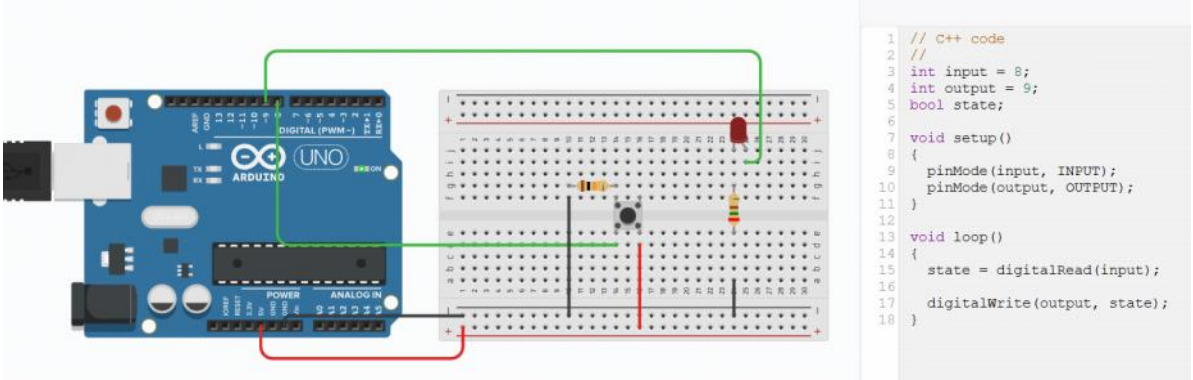
delay(задержка_в_миллисекундах);

// Подает аналоговую величину на пин (0-255)
analogWrite(led_pin, 0);
// Считывает значение с аналогового пина
int val = analogRead(led_pin);

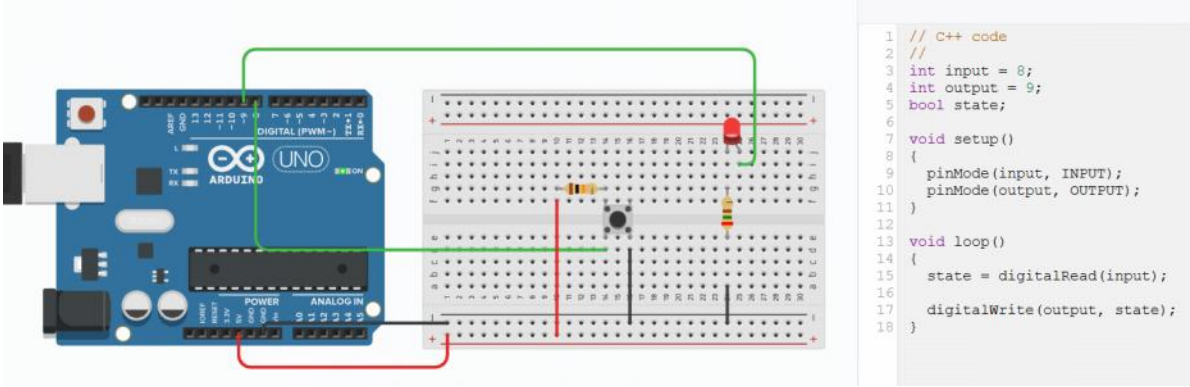
Токоограничивающий резистор (220-1k Ом)

tone(пин, частота)
noTone(пин)

Стягивающий резистор (Pull-Down)
соединяет вход с землей (GND) через резистор, когда источник сигнала отключен, вход остается в состоянии LOW. Стягивающий резистор должен иметь высокое сопротивление. (10кОм)



Подтягивающий резистор
соединяет вход с питанием (VCC) через резистор, когда источник сигнала отключен, вход остается в состоянии HIGH. Подтягивающий резистор должен иметь высокое сопротивление. (10кОм)



Arduino имеет аппаратную реализацию подтягивающих резисторов, включаемую при помощи INPUT_PULLUP

Программное подавление дребезга контактов

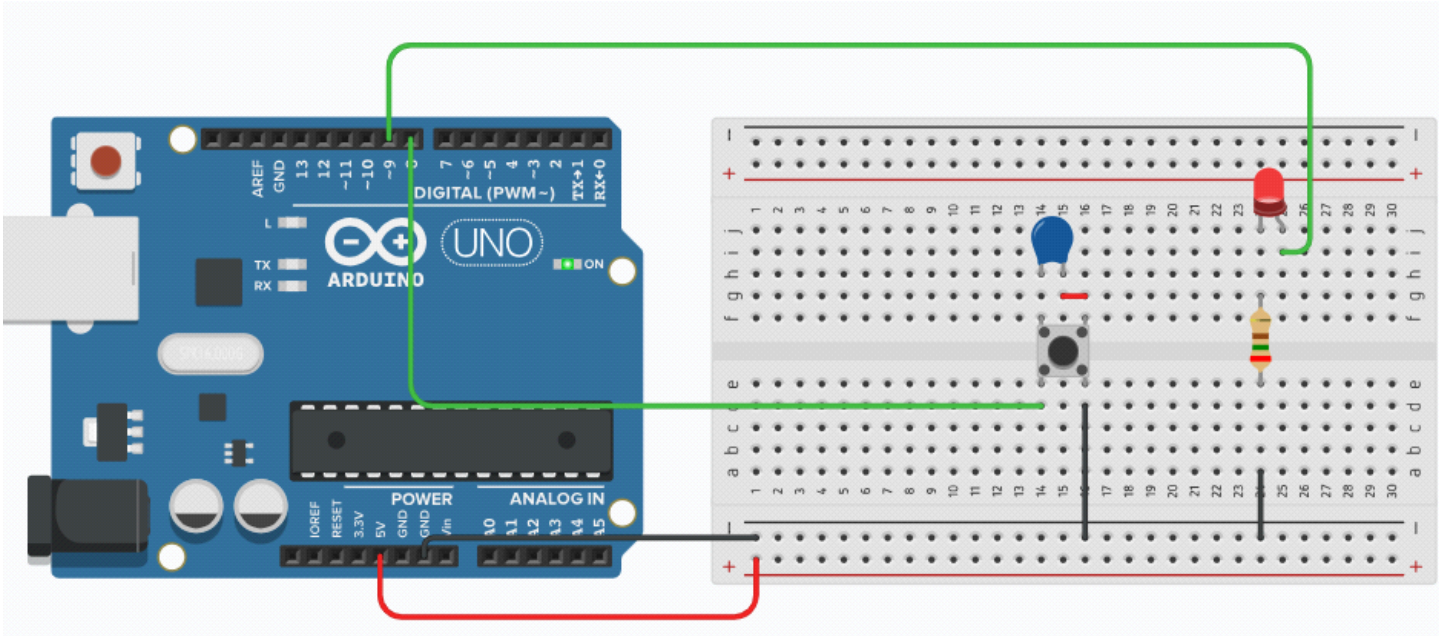
```
// C++ code
//
int input = 8;
int output = 9;
bool state;
bool prev;
void setup()
{
  pinMode(input, INPUT_PULLUP);
  pinMode(output, OUTPUT);
  Serial.begin(9600); //11520
}

void loop()
{
  state = digitalRead(input);

  if(state != prev){
    delay(10);
    state = digitalRead(input);
  }
  prev = state;

  digitalWrite(output, state);
}
```

Аппаратное подавление дребезга



Внешние прерывания (External hardware interrupt)

attachInterrupt(пин_прерывания, обработчик, режим)
Режимы:

- LOW – срабатывает когда на пине низкий сигнал
- CHANGE - при смене сигнала
- RISING - при смене с низкого на высокий
- FALLING - при смене с высокого на низкий

Arduino NANO поддерживает прерывания на

INT 0	INT 1
D2	D3

На всех пинах поддерживается PCINT - (Pin Change Interrupt), который не поддерживается в стандартной библиотеке, но может быть реализовано вручную или с помощью библиотек.

Основные платформы мобильных приложений

12 января 2026 г. 10:51

Android
IOS
Windows
Linux
RedOS M
Аврора ОС
HarmonyOS

НАТИВНЫЕ

Native – приложения, для создания которых используется язык программирования, IDE, средства разработки и другие инструменты для конкретной платформы, как правило предоставляемые поставщиками ОС

Android
Разработчик - Google
Доля рынка: ~70%
Язык программирования: Kotlin, Java
Среда разработки: Android Studio
Особенности:
- Открытый исходный код
- Широкий выбор устройств разных ценовых категорий
- Магазин приложений - Google Play Market
- Большое количество версий ОС, что приводит к фрагментации

IOS
Разработчик - Apple
Доля рынка: ~25%
Язык программирования: Swift, Objective-C
Среда разработки: Xcode
Особенности:
- Закрытая экосистема
- Поддержка ограниченного числа устройств (iPhone, iPad)
- Магазин приложений - AppStore
- Единообразие устройств и версий ОС, что облегчает тестирование

HarmonyOS
Разработчик - Huawei
Доля рынка: растет, особенно в Китае
Язык программирования: ArkTS, JavaScript, C++
Среда разработки: DevEco Studio
Особенности:
- Фокус на китайский рынок
- Магазин приложений - App Gallery
- Интеграция с умными устройствами (IoT)

Аврора ОС
Разработчик - Открытая мобильная платформа
Доля рынка: низкая (сконцентрирована в гос. структурах)
Язык программирования: C/C++
Среда разработки: Аврора SDK
Особенности:
- Фокус на защищенную инфраструктуру
- Ограниченная экосистема приложений
- Магазин приложений - RuStore
- Используется в гос. и корпоративных нуждах

КРОССПЛАТФОРМЕННЫЕ

Подход к разработке приложений, где используется единая кодовая база для разных ОС. Это достигается за счет использования фреймворков и технологий:

Flutter - разработан Google, язык Dart, высокая производительность и гибкость

React Native - разработаны Facebook, позволяет писать приложения на JS с использованием компонентов близких к нативным

MAUI - платформа Microsoft для разработки на C# + .NET
Avalonia и Uno Platform

Unity и др. игровые движки - предназначены для разработки игр, часто поддерживают множество платформ

Kotlin Multiplatform - фреймворк для разработки кроссплатформенных решений на Kotlin, особенно хорош, если есть кодовая база под Android

PWA – progressive web applications - преобразование веб-сайта в мобильное приложение

Ionic
Vue Storefront

Визуальные средства разработки

MIT App Inventor
Thunkable

Нативные
+ Высокая производительность и оптимизация под конкретные платформы
+ Полный доступ к функционалу устройства (GPS, датчики)
+ Лучшая адаптация пользователей
+ Высокая надежность и стабильность
- Работает только на одной платформе -> необходимо содержать команды разработки под разные платформы
- Сложность повторного использования кода -> Длительные сроки разработки
- Более высокая стоимость разработки

Кроссплатформенные
+ Единая кодовая база для всех платформ, что сокращает затраты
+ Более быстрая разработка и обновление
+ Больший охват аудитории
- Ограниченный доступ к некоторым функциям устройств
- Ниже производительность
- Возможные проблемы с адаптацией интерфейса под разные платформы

Основы языка Kotlin

14 января 2026 г. 9:08

Intellij Idea

.kt

Shift + F10 - запуск программы

Shift + F9 - отладка

Ctrl + F8 - точка останова

```
fun main() {
    println("Hello World!")
}
```

Main - точка входа

Функции

21 января 2026 г. 8:49

```
fun name(pram1:Int, param2:String = "Hello",
param3:Int) : Int {
    Тело функции
    return
}
```

Unit - аналог void в c#, используется в качестве возвращаемого, для функций, не возвращающих значение

Виды функций:

- Локальные - функции, объявленные внутри другой функции
- Однострочные - функция, объявленная как одно выражение
- Анонимные - функции, не имеющие имени
- Лямбда - выражения - сокращенная запись функций
- Функции высокого порядка - функции, которые либо принимают функцию в качестве параметра, либо возвращают функцию, либо и то, и другое
- Замыкание - объект функции, который запоминает окружение даже вне области своей видимости

Коллекции

26 января 2026 г. 11:14

Неизменяемые коллекции

Iterable - корневой класс коллекций
Collection

- size - количество элементов
- isEmpty() - проверка на пустоту
- contains(element) - содержит ли элемент
- containsAll(collection) - содержит ли все элементы

List - список
Set - множество
Map - набор ключ-значение (словарь)

Изменяемые коллекции

MutableIterable
MutableCollection

- add(element)
- remove(element)
- addAll(elements)
- removeAll(elements)
- clear() - очистка списка

MutableList, MutableSet, MutableMap

Операции с коллекциями

all(predicate: (T) -> Boolean): Boolean
- все элементы соответствуют предикату

any(): Boolean
any(условие): Boolean
- коллекция содержит хотя бы 1 элемент, который удовлетворяет условию

average() : Double
- среднее значение числовой коллекции

chunked(size): List<List<T>>
расщепляет коллекцию на список списков по size

элементов

count(): Int
count(условие): Int
- количество элементов, которые удовлетворяют условию

distinct(): List<T>
- новая коллекция, содержащая только уникальные элементы

distinctBy(selector): List<T>
- новая коллекция, содержащая только уникальные элементы с учетом селектора

drop(количество): List<T>
- новая коллекция, содержащая элементы за исключением первых n

dropWhile(условие): List<T>
- новая коллекция, содержащая элементы за исключением первых соответствующих условию

elementAt(index): T
- элемент по индексу
elementAtOrElse(index, default): T
elementAtOrNull(index): T?

filter(условие): List<T>
возвращает коллекцию элементов, соответствующих условию
filterNot(условие): List<T>
возвращает коллекцию элементов, НЕ соответствующих условию
filterNotNull(): List<T>
возвращает элементы не null

find(условие): T?
первый элемент, соответствующий условию или null
findLast(условие): T?
последний элемент, соответствующий условию или null

first(): T

первый элемент коллекции
first(условие): T
первый элемент, соответствующий условию или исключение

firstOrNull(): T?
первый элемент коллекции или null
firstOrNull(условие): T?
первый элемент, соответствующий условию или null

fold(начальное значение, operation: (R,T) -> R): R
Значение результата функции над каждым элементом

forEach(действие)
Выполняет действие для каждого элемента коллекции

groupBy(селектор):Map<K, List<T>>
Группирует элементы по ключу

indexOf(элемент): Int
Индекс первого вхождения или -1

indexOfFirst(условие): Int
Индекс первого вхождения или -1

indexOfLast(условие): Int
Индекс последнего вхождения или -1

intersect(другая коллекция): Set
Все элементы которые есть и в этой и в другой коллекции

joinToString(): String
Генерирует строку из коллекции

last():T
Последний элемент
last(условие):T
Последний элемент соответствующий условию
lastOrNull():T?
Последний элемент или null
lastOrNull(условие):T?
Последний элемент соответствующий условию или

null

lastIndexOf(элемент): Int
Индекс последнего вхождения или -1

map(преобразование): List<T>
применяет к коллекции функцию преобразования
mapIndexed(преобразование): List<T>
применяет к коллекции функцию преобразования,
имеет доступ к индексу
mapNotNull(преобразование): List<T>
применяет к коллекции функцию преобразования, и
возвращает элементы, не равные Null

maxOf(условие):T
максимальное на основе условия
minOf(условие):T
минимальное на основе условия

minus(элемент):List<T>
plus(элемент):List<T>

reduce(operation: (R,T) -> R):R
Значение результата функции над каждым элементом

shuffled():List<T>
Перемешивает коллекцию

sorted():List<T>
sortedDescending():List<T>
sortedBy(условие):List<T>
sortedByDescending(условие):List<T>

sum(): Int
сумма элементов коллекции
sumOf(условие): Int

subtract(другая коллекция): Set
Элементы, которые есть в текущей и отсутствуют в
другой

take(количество):List<T>
первые n элементов
takeWhile(условие):List<T>

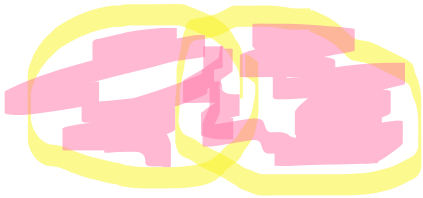
берем, пока выполняется условие

union(другая коллекция): Set
Все элементы обеих коллекций

List - Неизменяемый список listOf()

MutableList – mutableListOf()
ArrayList – arrayListOf()

add(элемент)
addAll(коллекция)
remove(элемент)
removeAt(индекс)
clear()



Set – неизменяемое множество setOf()

MutableSet – mutableSetOf()

HashSet - хэш-таблица hashSetOf()
LinkedHashSet - хэш-таблица + связный список
linkedSetOf()

Map - неизменяемый словарь mapOf(1 to "qwe", 2 to "asd")

containsKey()
containsValue()

HashMap – hashMapOf()
LinkedHashMap – linkedMapOf()

put(ключ, значение)
putAll(словарь)
set(ключ, значение)
remove(ключ)

Sequence - последовательность

generateSequence

Строки

4 февраля 2026 г. 8:48

String

Корутины

4 февраля 2026 г. 9:33

kotlinx-coroutines-core-jvm

Корутина - блок кода, который может выполняться параллельно с основным кодом (легковесный поток)

Параллельные вычисления - выполнение нескольких задач одновременно

Асинхронность позволяет не блокировать основной поток при выполнении задачи, занимающей много времени

Suspend - функция, которая может приостанавливать свое выполнение и возобновлять его через некоторый период времени

Контекст корутины (coroutineScope) - область видимости, содержащая контекст для настройки и отмены корутины.

launch - построитель корутин, создает корутину в виде блока кода и запускает ее параллельно с кодом

runBlocking – контекст корутины, блокирующий главный поток

Job - объект возвращаемый функцией launch, позволяющий управлять запущенной корутиной

job.join() - ожидание завершения
job.start() - запуск
job.cancel() - отмена
job.cancelAndJoin() - отмена и ожидание

Deferred – объект, ждущий результат, наследник Job

Эмуляторы

9 февраля 2026 г. 12:16

Эмулятор - ПО, обеспечивающее эмулирование поведения одной ОС в другой.

- VirtualBox
- BlueStacks
- NoxPlayer
- QEMU

Создание эмулятора

1. Виртуальная машина в VirtualBox(VmWare) с установленным Android
   https://sourceforge.net/projects/android-x86/files/Release%209.0/android-x86_64-9.0-r2.iso/download

2. Android Studio
   Device Manager – Add new device

3. VisualStudio
   Диспетчер устройств Android

   Android Studio использует QEMU для эмуляции устройств

   AVD (Android Virtual Device)

   + достаточно реалистичное представление

устройства
- необходимость моделирования всех аппаратных устройств сказывается на производительности

Как ускорить работу

- Аппаратное ускорение (Hyper-V, KVM, HAXM)
- Использование снапшотов (снимки текущего состояния ВМ) для восстановления состояния без загрузки
- Использовать реальное устройство

ADB (Android Debug Bridge) - консольная утилита для отладки Android-устройств
Android/SDK/platform-tools

adb help справка

adb connect адрес:порт
adb devices список устройств
-l подробная информация

adb kill-server завершение процесса сервера
adb start-server запуск процесса сервера

ADB - клиент-серверное приложение
Клиент на ПК разработчика cmd -> adb
adbd (adb daemon) - процесс, который запускает команды на устройстве

5554-5584 - четные порты под эмуляторы,

нечетные - под устройства

adb [-s SERIAL|-d|-e] команда

reboot перезапуск

root подключение с правами суперпользователя

install имя_файла.apk

uninstall имя_пакета

push источник устройство

(adb push C:\Temp\1.txt \sdcard\)

pull устройство приемник

(adb pull \sdcard\1.txt C:\Temp\)

shell выполнение команды в терминале устройства

ls список файлов и папок

ls /system/bin список команд

pm list packages список пакетов

screencap –p /sdcard/example.png скриншот

screenrecord /sdcard/example.mp4 запись экрана (Ctrl+C в терминале для завершения)

logcat - логи устройства

-s имя_пакета логи по пакету

Создание мобильного приложения

11 февраля 2026 г. 9:51

Jetpack Compose - набор инструментов для создания современных интерфейсов пользователя в приложениях на Android (2019)
Он использует декларативный подход (без XML-макета). Элементы управления создаются в коде, там же указываются их настройки

EmptyActivity - базовый пустой проект Jetpack Compose

Структура проекта

- manifests
  - AndroidManifest.xml - свойства приложения, его разрешения
- kotlin+java - код приложения
  - com.example.myapplication
    - ui.theme
      - Color.kt - цвета
      - Theme.kt - цветовая схема
      - Type.kt - шрифты
    - MainActivity.kt - главная активность (окно) приложения
  - (test) - модульные тесты приложения
  - (androidTest) - инструментальные тесты
- res - ресурсы
  - drawable - изображения
  - mipmap - иконки

    - values - ресурсы в строковом виде (цвета, значения, строки)
    - xml - другое
- Gradle Scripts
    - build.gradle.kts - настройки сборки, библиотеки, зависимости
    - libs.versions.toml

MainActivity

пакет package

библиотеки import

MainActivity - наследник класса ComponentActivity
onCreate - функция, вызываемая при инициализации приложения. Параметр Bundle хранит состояние Activity

setContent - определяет интерфейс приложения с помощью компонентов Composable

MaterialTheme

Scaffold - стандартный корневой контейнер андройд-приложения

@Composable - определяет функцию, описывающую визуальный интерфейс приложения (может быть вызвана внутри

Composable - контекста)

@Preview - позволяет предпросмотр для @Composable-функций

Layout

Box - Контейнер для абсолютного позиционирования и позиционирования относительно коробки

Column - Контейнер для расположения элементов в колонку

Row - контейнер для расположения элементов в строку

ConstraintLayout

BoxWithConstraints

Foundation

BaseTextField - простейшее поле ввода

Canvas - холст

Image - картинка

LazyColumn

LazyRow - списки с ленивой загрузкой

Shape - фигура

Text - надпись

MaterialDesign - компоненты разработанные для соответствия стилю Google Material

Scaffold - контейнер

Button - кнопка

Card - карточка
CheckBox - флажок
RadioButton - переключатель
DropdownMenu -выпадающий список
Switch - выключатель
Slider - ползунок
TextField - поле ввода
AlertDialog - всплывающее диалоговое окно

Модификаторы - функции, которые задают отдельные аспекты для компонентов (модифицируют внешний вид компонента), например устанавливают размер, фон и т.д.

Основные платформы мобильных приложений
12 января 2026 г. 0:08

Windows
Linux
Android
IOS
RedOS
Аврора ОС

Нативные

Native
Приложения, для создания которых используется язык, средства разработки, и другие инструменты для одной ОС, обычно предоставляемые поставщиками ОС

Android
Разработчик - Google
Доля рынка: ~70%
Язык программирования: Java, Kotlin
IDE: Android Studio + Android SDK

Широкий выбор устройств разных ценовых категорий
Открытый исходный код
Google Play
Большая фрагментация версий ОС

IOS
Разработчик - Apple
Доля рынка: ~25%
Язык программирования: Swift, Objective-C
IDE: Xcode

Ограниченный круг устройств (iPhone, iPad)
Закрытая экосистема
App Store
Единообразие устройств и версий ОС

HarmonyOS
Разработчик: Huawei
Доля рынка: растет, особенно в Китае
Язык программирования: ArkTS, JS, C/C++
IDE: DevEco Studio

Фокус на китайский рынок
AppGallery
Интеграция с умными устройствами (IoT)

Аврора ОС
Разработчик: Открытая мобильная платформа
Доля рынка: низкая (сконцентрирована в гос. структурах)
Язык программирования: C/C++
IDE: Аврора SDK

Фокус на защищенную инфраструктуру
Ограниченная экосистема приложений
Rustore
Используется в гос. и корпоративных нуждах

Кроссплатформенные

Подход к разработке приложений, где используется единая кодовая база для разных ОС, что достигается за счет применения специальных инструментов и фреймворков

Flutter - разработан Google, язык Dart, популярен, благодаря высокой производительности и гибкости

ReactNative - от Facebook, позволяет писать на JS с использованием компонентов, близких к нативным

MAUI - от Microsoft, разработка на C# с .NET
Avalonia - Решение сообщества для C#

Unity - используется для игр, обеспечивает поддержку множества платформ

Ionic - гибридные приложения с использованием веб-технологий (HTML, CSS, JS)

Kotlin Multiplatform - хорош, когда есть кодовая база на Android

PWA – Progressive Web Application
Позволяет преобразовать веб-сайт в мобильное приложение

Доступ к возможностям устройства (не ко всем, ограниченный)
Преимущества веб-приложений
Может работать офлайн
Увеличенный расход энергии

Плюсы и минусы

Нативные

+ Высокая производительность
+ Полный доступ к функционалу
+ Лучшая адаптация пользователя
+ Высокая надежность и стабильность

- Высокая стоимость разработки
- Больший срок разработки
- Необходимость содержать команды для разных платформ
- Сложность повторного использования кода

Кроссплатформенные

+ Единая кодовая база для всех платформ, что сокращает затраты времени и средств
+ Быстрая разработка и обновление
+ Возможность охватить большую аудиторию, выпуская приложение сразу на нескольких платформах

- Ограниченный доступ к функциям устройства
- Ниже производительность
- Проблемы с адаптацией интерфейса под разные платформы

Платформы для визуальной разработки приложений

MIT App Inventor
Thunkable

Основы языка Kotlin

12 января 2026 г. 12:41

IDE: InteliJ Idea (GigaIDE)

.kt

Ctrl+F8 - точка останова
Shift + F9 - отладка
Shift + F10 - запуск

Точка входа

```
fun main() {
    println("Hello World!")
}
```

Функции

20 января 2026 г. 11:48

Локальные - функции, объявленные внутри другой функции

Однострочные - функция в виде одного выражения

Анонимные функции - объявляются как обычные, но без имени

Лямбда-выражения - сокращенная запись функции

Функции высокого порядка - функции, которые либо принимают функцию в качестве параметра, либо возвращают функцию, либо делают и то, и другое

Замыкания - объект функции, который запоминает свое окружение даже вне своей области видимости

ООП

26 января 2026 г. 10:10

Виды классов
data class - для хранения данных
final class - запрещает наследование, по умолчанию
open class - разрешает наследование
sealed class - запечатанный
abstract class - абстрактный
enum class - перечисления

Scope-функция -функция, которая выполняет блок кода для объекта или контекста, используются, что бы делать код компактнее

Коллекции

3 февраля 2026 г. 8:46

Iterable
Collection

Immutable - коллекции (неизменяемые)

- size - количество элементов
- isEmpty() - true, если пустая
- contains(элемент) - если коллекция содержит элемент
- containsAll(коллекция) - если коллекция содержит все элементы

List - список элементов
Set - множество
Map - коллекция ключ-значение (словарь)

MutableIterable
MutableCollection

Mutable - коллекций (изменяемые)

add(элемент) - добавление элемента
remove(элемент) - удаление элемента
addAll(элементы) - добавление набора элементов
removeAll(элементы) - удаление набора элементов
clear() - удаление всех элементов

MutableList
MutableSet
MutableMap

Операции с коллекциями

all(условие):Boolean
Если все элементы соответствуют условию

any():Boolean
Если в коллекции есть хотя бы один элемент
any(условие):Boolean
Если хотя бы 1 элемент соответствуют условию

average(): Double
Вычисляет среднее, работает только для числовых коллекций

chuncked(size:Int):List<List<T>>
расщепляет коллекцию на список, состоящий из списков элементов начальной коллекции поделенных по size штук

count():Int
Количество элементов в списке
count(условие):Int
Количество элементов, удовлетворяющих условию

distinct(): List<T>
Только уникальные элементы
distinctBy(селектор): List<T>
Только уникальные элементы с учетом селектора

drop(n)/dropWhile(условие):List<T>
возвращает новую коллекцию, за исключением первых n / первых соответствующих условию

take(n)/takeWhile(условие):List<T>
возвращает новую коллекцию, содержащую первые n / первые соответствующие условию

elementAt(index):T
элемент по индексу, если индекса нет -> исключение
elementAtOrElse(index, default):T
элемент по индексу, если индекса нет -> default
elementAtOrNull(index):T
элемент по индексу, если индекса нет -> null

filter(условие):List<T>
Элементы удовлетворяющие условию
filterNot(условие):List<T>
Элементы НЕ удовлетворяющие условию
filterNotNull():List<T>
Элементы которые не равны null

find(условие):T
первый элемент удовлетворяющий условию
findLast(условие):T
последний элемент удовлетворяющий условию

first()/first(условие):T
первый элемент / первый удовлетворяющий условию
при отсутствии элемента бросает исключение
firstOrNull()/firstOrNull(условие):T?
первый элемент / первый удовлетворяющий условию
или null

last()/last(условие):T
последний элемент / последний удовлетворяющий условию
при отсутствии элемента бросает исключение

lastOrNull()/lastOrNull(условие):T?
последний элемент / последний удовлетворяющий условию или null

indexOf(элемент):Int
индекс первого вхождения
indexOfFirst(условие):Int
индекс первого элемента соответствующего условию
indexOfLast(условие):Int
индекс последнего элемента соответствующего условию

groupBy(селектор): Map<K, List<T>>
Группировка по ключу

forEach(действие)
действие для каждого элемента

joinToString():String
соединяет коллекцию в строку

map(преобразование):List<T>
применяет преобразование для всех элементов коллекции
mapIndexed(преобразование)
применяет преобразование для всех элементов коллекции, имеет доступ к индексу элемента

maxOf(селектор)/minOf(селектор)

minus(элемент)
plus(элемент)

reduce(operation: (S,T)->S):S

возвращает значение, являющееся результатом действия функции над каждым элементом

shuffled():List<T>
перемешивает коллекцию

sorted():List<T>
сортирует коллекцию
sortedBy(селектор):List<T>
сортирует коллекцию с учетом селектора

sortedDescending():List<T>
сортирует коллекцию по убыванию
sortedByDescending(селектор):List<T>
сортирует коллекцию по убыванию с учетом селектора

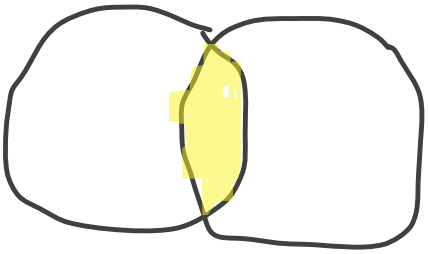
intersect(коллекция): Set
пересечение множеств
subtract(коллекция): Set
вычитание множеств
union(коллекция): Set
объединение множеств

sum():Int
сумма элементов

List – listOf()

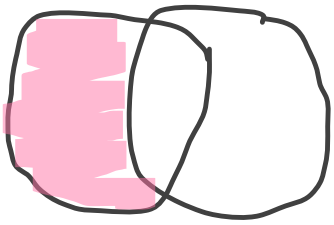
MutableList – mutableListOf()
ArrayList – arrayListOf()

Set – setOf()



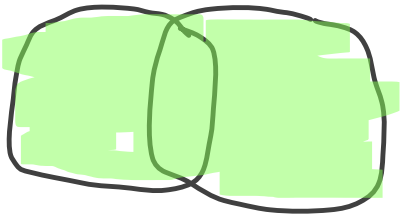
MutableSet – mutableSetOf()
LinkedHashSet – linkedSetOf()
HashSet() - hashSetOf()

Map – mapOf(*key* to *value*, *key* to *value*)



MutableMap – mutableMapOf()

HashMap – hashMapOf()
LinkedHashMap – linkedMapOf()



Sequence – sequenceOf()

generateSequence
sequence

Строки

4 февраля 2026 г. 10:49

String

Строки неизменяемые
Представляют собой массив символов (Char)

Корутины

4 февраля 2026 г. 11:25

File – Project Structure – Libraries - "+" - from Maven - kotlinx-coroutines-core-jvm

Корутина - блок кода, который может выполняться параллельно с основным кодом (легковесный поток)

Параллельные вычисления - это выполнение нескольких задач одновременно

Асинхронность - предоставляет возможность не блокировать основной поток во время выполнения задачи, затрачивающей много времени

Suspend - функция которая может приостанавливать свое выполнение и возобновлять его не блокируя поток

Контекст корутины (coroutineScope) - область видимости, которая содержит контекст для настройки, запуска и отмены корутин.
Создается в suspend-функции
Создает контекст и дожидается выполнения всех определенных внутри корутин

launch - построитель корутины, создает корутину в виде блока кода и запускает ее параллельно с остальным кодом

Внутри launch могут быть другие launch

runBlocking - блокирует вызывающий поток

Job - объект задачи, возвращаемый функцией launch
job.join() - ожидание задачи

Диспетчер корутины - поток, в котором будет выполняться корутина
Dispatchers.IO - для операций ввода-вывода
Dispatchers.Main - для запуска в основном потоке (UI)
Dispatchers.Default - для вычислений требующих ресурсы
Dispatchers.Unconfined - не определен

Deffered – объект задачи, возвращающий значение, возвращаемый функцией async

deffered.await() - получение значения

Эмуляторы

4 февраля 2026 г. 14:04

Эмулятор - ПО, обеспечивающее эмулирование поведения одной системы в другой

Популярные эмуляторы:

- BlueStacks
- NoxPlayer
- QEMU

Создание эмулятора:

- Создание виртуальной машины (VirtualBox/VMWare) и установка Android [android-x86 - Browse /Release 9.0 at SourceForge.net]

- AndroidStudio - устройства

- VisualStudio – Диспетчер устройств Android

AVD (Android Virtual Device) - > QEMU

+ более реалистичное представление устройства
- необходимо моделировать все аппаратные компоненты, что нагружает систему

- Использование снапшотов (снимки текущего состояния эмулятора) для восстановления состояния без прохождения загрузки
- Использование аппаратного ускорения (Hyper-V,

KVM, HAXM)
- Использовать реальное устройство

ADB (Android Debug Bridge)
adb.exe -> SDK\platform-tools\

ADB - клиент-серверное приложение для доступа к эмуляторам или устройствам

1. Клиент работает с пк и отправляет команды cmd -> adb
2. adbd (adb daemon) - процесс запускает команды на устройстве, работает в фоновом режиме

Порты для подключения
5037 – TCP
5554-5584 (четные) - 16 эмуляторов
5555-5585 (нечетные) - для остальных устройств

adb devices список доступных устройств
-l подробный вывод информации
adb connect адрес:порт
adb help
adb kill-server - завершает процесс сервера adbd
adb start-server - включает adbd
adb –s emulator-5555 install hello.apk
adb –s адрес:порт reboot
adb push источник устройство - копирование файлов на телефон
adb pull устройство приемник - копирование файлов с телефона

adb uninstall имя_пакета - удаление пакета
adb shell - терминал
    ls - список файлов и папок
    ls /system/bin - список всех команд
    pm list packages - список всех установленных пакетов на устройстве
    screencap –p /sdcard/screen.png - скриншот
    screenrecord /sdcard/screen.mp4 - запись экрана. Ctrl+C в консоли для завершения записи

adb logcat - лог действий

Мобильные приложения

13 февраля 2026 г. 12:44

Empty Activity
Jetpack Compose - набор инструментов для создания современных интерфейсов пользователя в приложениях Android (2019)
Использует декларативный подход (верстка без XML макета). Все элементы управления указываются в коде, вместе с их настройками.

Android

app

- manifests
  - AndroidManifest.xml - файл конфигурации проекта, в нем указываются компоненты приложения, его разрешения
- kotlin+java - код приложения
  - com.example.myapplication
    - ui.theme
      - Color.kt - цвета
      - Theme.kt - цветовые схемы
      - Type.kt - шрифты
    - MainActivity - главная активность приложения
  - (test) - юнит-тесты приложения
  - (androidTest) - инструментальные тесты приложения
- res - ресурсы приложения

- drawable - изображения
- mipmap - иконки
- values - строковые ресурсы (цвета, строки, значения)
- xml - другие

Gradle Scripts - настройки Gradle

- build.gradle.kts (Module) - настройки и подключения зависимостей
- build.gradle.kts (Project) - настройки всего проекта
- libs.versions.toml - версии библиотек

MainActivity - наследник класса ComponentActivity

onCreate - вызывается при инициализации приложения

Принимает Bundle - сохраненная информация о состоянии Activity

setContent - определяет интерфейс приложения на экране (отрисовывает Composable).

@Composable - обозначает функцию, описывающую визуальный интерфейс приложения

ApplicationTheme - Composable-функция, определяет тему приложения для вложенных элементов

Scaffold - Composable-функция, определяющая контейнер для стандартного интерфейса приложения и применяющая стиль.

@Preview - обозначает функцию,позволяющую предпросматривать интерфейс без запуска приложения

Основные компоненты
Layout:

- Box - коробка, позволяет позиционировать внутри содержимое
- Column - колонка, позволяет располагать элементы вертикально друг за другом
- Row - строка, позволяет располагать элементы горизонтально друг за другом
- ConstraintLayout
- BoxWithCostraints

Foundation:

- Canvas - холст
- Image - Изображение
- Shape - фигура
- Text - надпись
- BaseTextField - базовое поле ввода
- LazyColumn
- LazyRow - ленивые списки (загружаются по мере необходимости)

Material Design - компоненты соответствующие

рекомендациям по визуальному стилю Google Material

- AlertDialog - всплывающее диалоговое окно
- Button - кнопка
- Card - карточка
- CircularProgressIndicator - круглый прогрессбар
- DropDownMenu - выпадающий список
- CheckBox - флажок
- FloatingActionButton - плавающая кнопка
- LinearProgerssIndicator - прогрессбар
- RadioButton - переключатель
- Scaffold - контейнер
- Slider - ползунок
- Snackbar - всплывающее сообщение
- Switch - выключатель
- TextField - текстовое поле ввода

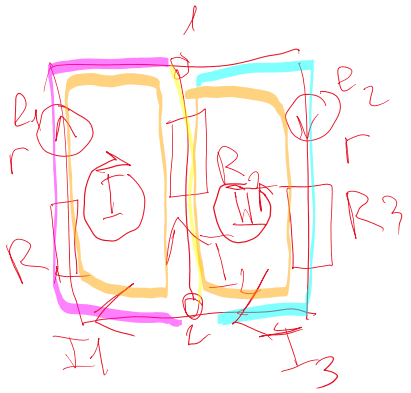
16 января 2026 г. 10:52

1) проверить наличие → при отсутствии заказать (реализовано в отдельном модуле)
2) Взять яйцо и масло

4) Взять сковороду
5) Включить плиту
6) Поставить сковороду на конфорку
7) Налить в сковороду масло
8) Разбить яйцо в сковороду
9) Дождаться готовности
10) Выключить плиту

Физика

22 января 2026 г. 14:30



алгебраическая сумма токов сходящихся в узле равна 0 $\sum I_i = 0$

$$\sum \mathcal{E}_i = \sum I_i R_i$$

находим узлы (сколько узлов – столько уравнений)

считаем ветки 3 ветки = 3 тока

I_1, I_2, I_3

1 узел

$$I_1 + I_2 - I_3 = 0$$

2

$$-I_1 - I_2 + I_3 = 0$$

Найти замкнутые контуры

1 контур I_1 по + I_2 против – с учетом

$$\mathcal{E}_1 = I_1(R_1 + r) - I_2 \cdot R_2$$

2 контур

$$-\mathcal{E}_2 = -I_3(R_3 + r) - I_2 R_2$$